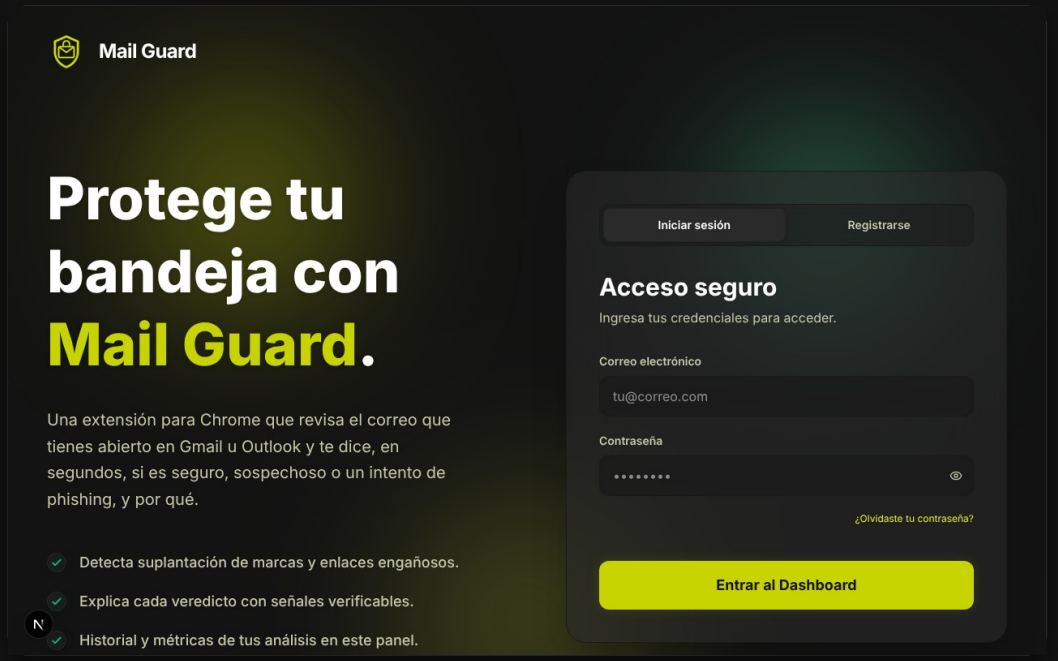
Marcos de Trabajo · Ing. Carlos Pineda

Mail Guard

Microservicios y pruebas de software: unitarias, de API, de seguridad y de aceptación



Extensión + panel web · 3 microservicios · 128 pruebas

uts-mail-guard-extension-dashboard.vercel.app

El problema y la solución

El problema

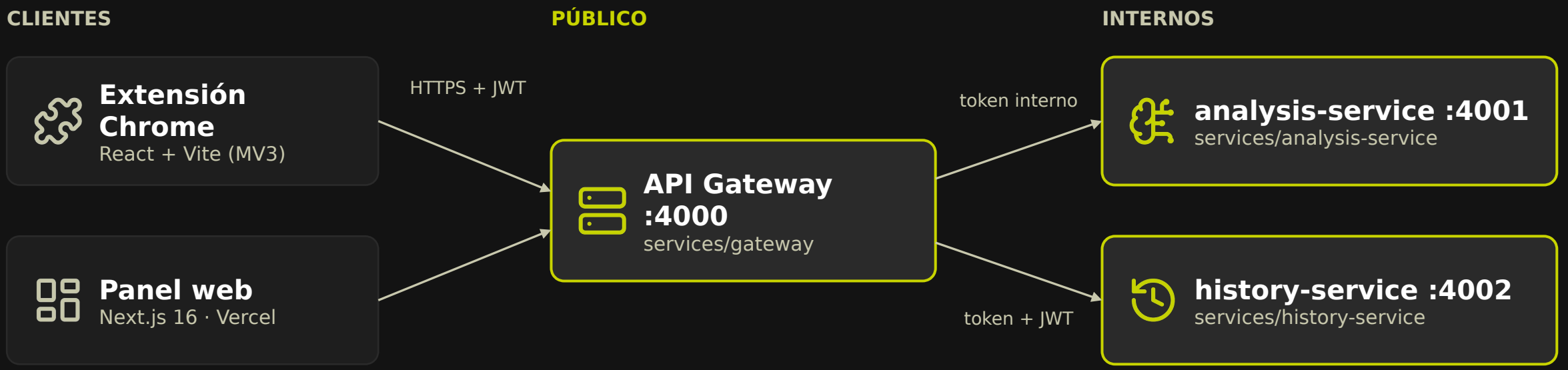
- Los correos de phishing imitan a bancos y servicios conocidos.
- Usan urgencia: «tu cuenta será bloqueada en 24 horas».
- Piden contraseñas o datos de tarjeta, o esconden enlaces peligrosos.

La solución: Mail Guard

- Extensión de Chrome que lee el correo abierto y lo analiza en segundos.
- Explica el veredicto con señales verificables: remitente, contenido y enlaces.
- Panel web con historial y métricas para usuarios registrados.

Seguro **Sospechoso** **Peligroso**

Arquitectura: tres microservicios detrás de un gateway



SERVICIOS EXTERNOS

Supabase
Auth + Postgres con RLS

Jev (TypeSafe AI)
Motor de decisión

Upstash Redis
Caché y cuotas (opcional)

Solo el gateway es público; los servicios internos rechazan cualquier llamada sin el token interno.

Los tres microservicios: dónde están y qué hacen

API Gateway

`services/gateway · puerto 4000`

- Único punto de entrada público
- Verifica el JWT de Supabase
- Controla la cuota diaria de análisis
- Orquesta: pide el análisis y, con sesión, lo guarda
- Errores uniformes: { error }

analysis-service

`services/analysis-service · puerto 4001`

- Extrae evidencias: dominios, enlaces, urgencia
- Motor de decisión: Jev, un LLM o reglas locales
- Mezcla 70 % modelo + 30 % reglas
- Caché por huella SHA-256 del correo
- Si la IA falla, responde con reglas

history-service

`services/history-service · puerto 4002`

- Guarda análisis, perfil y métricas
- Supabase con RLS: cada quien ve lo suyo
- No duplica: (usuario, huella) es único
- Sincroniza análisis hechos sin conexión
- Permite borrar todo el historial

Dónde está cada cosa en el repositorio

`apps/extension-web`	Extensión de Chrome
`apps/extension-dashboard`	Panel web (Next.js)
`services/gateway`	API Gateway
`services/analysis-service`	Análisis de phishing
`services/history-service`	Historial y métricas
`packages/contracts`	Contratos Zod compartidos
`packages/service-kit`	Base común de los servicios
`packages/ui · api-client`	Interfaz y cliente HTTP
`supabase/migrations`	Esquema SQL y políticas RLS
`docs/`	Arquitectura, casos de uso, Postman

Por dentro: arquitectura hexagonal

`domain/`	Reglas puras: evidencias, puntajes
`application/`	Casos de uso + puertos (interfaces)
`infrastructure/`	Adaptadores: Jev, Redis, Supabase
`http/`	Rutas Hono (sin lógica de negocio)
`test/`	unitarias · api/ · security/ · acceptance/

Recorrido de un análisis, paso a paso

1

El usuario pulsa «Escanear correo»

Extensión

2

La extensión lee remitente, asunto, texto y enlaces

Extensión

3

El gateway verifica el JWT y la cuota diaria

Gateway

4

Caché → evidencias → motor de IA → veredicto

analysis-service

5

Con sesión, se guarda en el historial (RLS)

history-service

6

Se muestra el veredicto, las señales y un consejo

Extensión

Un resultado en caché no consume cuota; si guardar falla, el análisis igual se muestra y la extensión lo reintenta después.

Seguridad desde el diseño

Autenticación

El usuario inicia sesión en Supabase y recibe un JWT. El gateway valida su firma con getClaims antes de dar acceso.

Autorización

Row Level Security en Postgres: cada fila tiene dueño y la base solo devuelve las del usuario del JWT.

Entre microservicios

Token interno en la cabecera x-internal-token, comparado en tiempo constante. Sin él, 401.

Protección contra abuso

Cuota diaria (5 sin cuenta, 100 con cuenta), cuerpo máximo de 512 KB y cabeceras seguras.

Despliegue en la nube

Render

Los 3 microservicios

`mail-guard-gateway.onrender.com`

Vercel

Panel web (Next.js)

`uts-mail-guard-extension-dashboard.vercel.app`

Supabase

Usuarios + base de datos

`Postgres con RLS`

Chrome

Extensión

`Se carga desde chrome://extensions`

Plan gratuito: Render duerme los servicios tras 15 min sin uso. Antes de la demo hay que abrir el /health de cada servicio y esperar a que /health/services muestre los tres en «ok».



Segunda parte

Pruebas de software

Cuatro tipos de prueba, 128 casos y todos con evidencia en el repositorio

Estrategia: cuatro tipos de prueba

Tipo	Qué valida	Herramienta en Mail Guard	Visto en clase
Unitarias	Una función o caso de uso aislado	bun test	Jasmine · JUnit · pytest
API	Endpoints HTTP reales entre los 3 servicios	bun test + Postman / Newman	Postman
Seguridad	Autenticación, autorización y abuso	bun test + Supabase real	Autenticación y autorización
Aceptación	Casos de uso en Dado / Cuando / Entonces	Gherkin (formato Cucumber)	Cucumber

¿Y Selenium o Appium? No hay app móvil (Appium no aplica) y las pruebas se centran en los microservicios; automatizar la extensión con Selenium queda como trabajo futuro.

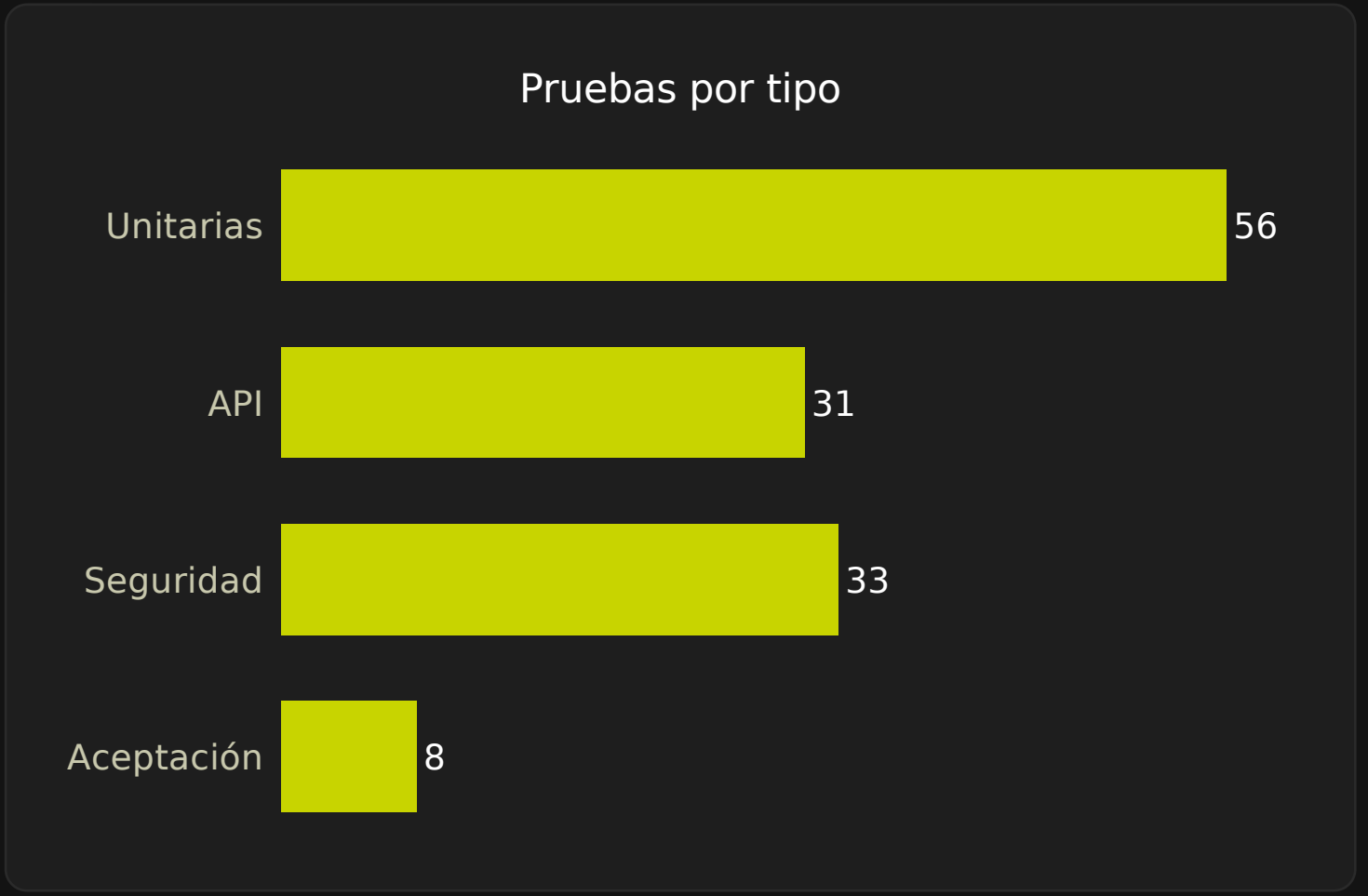
Resultados: 128 de 128 pruebas pasan

128/128 pruebas pasan

0 fallas

251 aserciones (expect)

17 archivos de prueba



13 de las 31 pruebas de API se ejecutan con Postman/Newman contra el gateway desplegado en Render.

Pruebas unitarias · 56

Prueban cada pieza por separado, reemplazando red y base de datos por dobles.

`scoring.test.ts`

Puntaje y veredicto: phishing → peligroso

`evidence.test.ts`

Dominios que imitan marcas, IPs, acortadores

`analyze-email.test.ts`

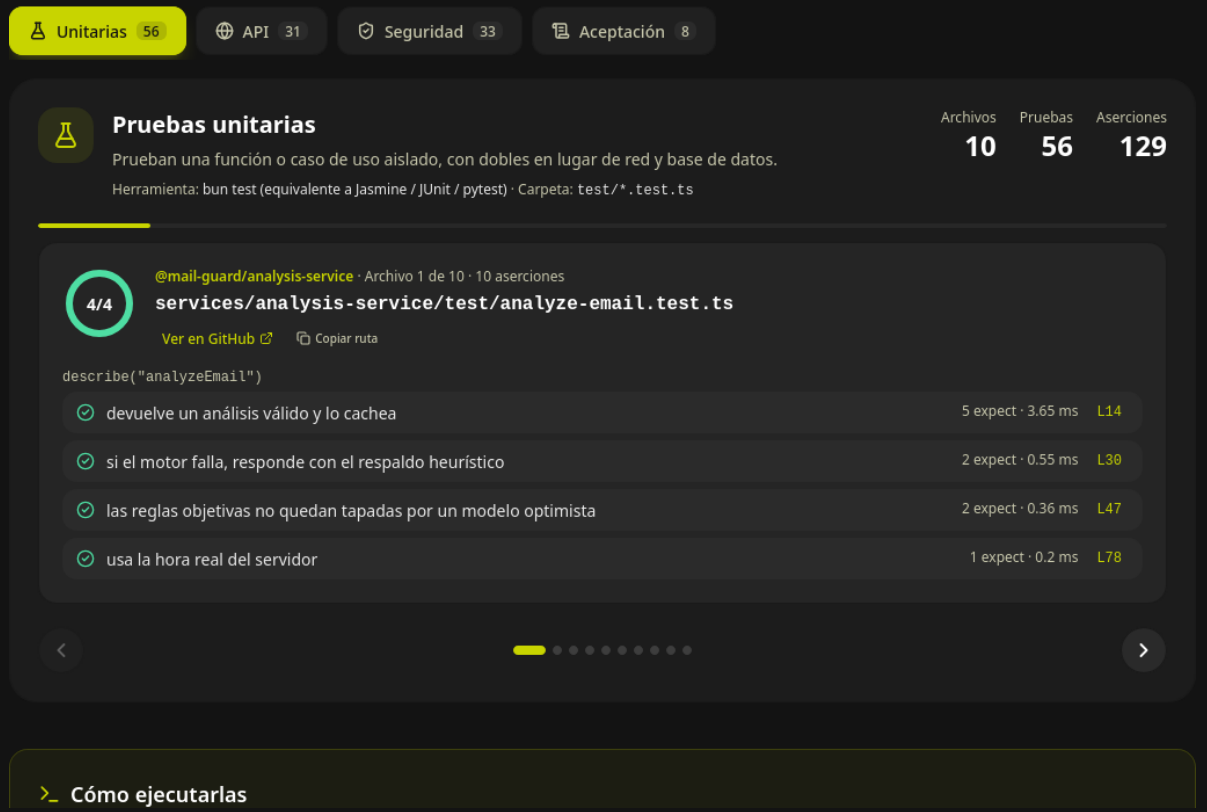
Caché y respaldo cuando la IA falla

`gateway.test.ts`

Cuota, guardado con JWT, validación

`use-cases.test.ts`

Historial: deduplicar, métricas, borrar



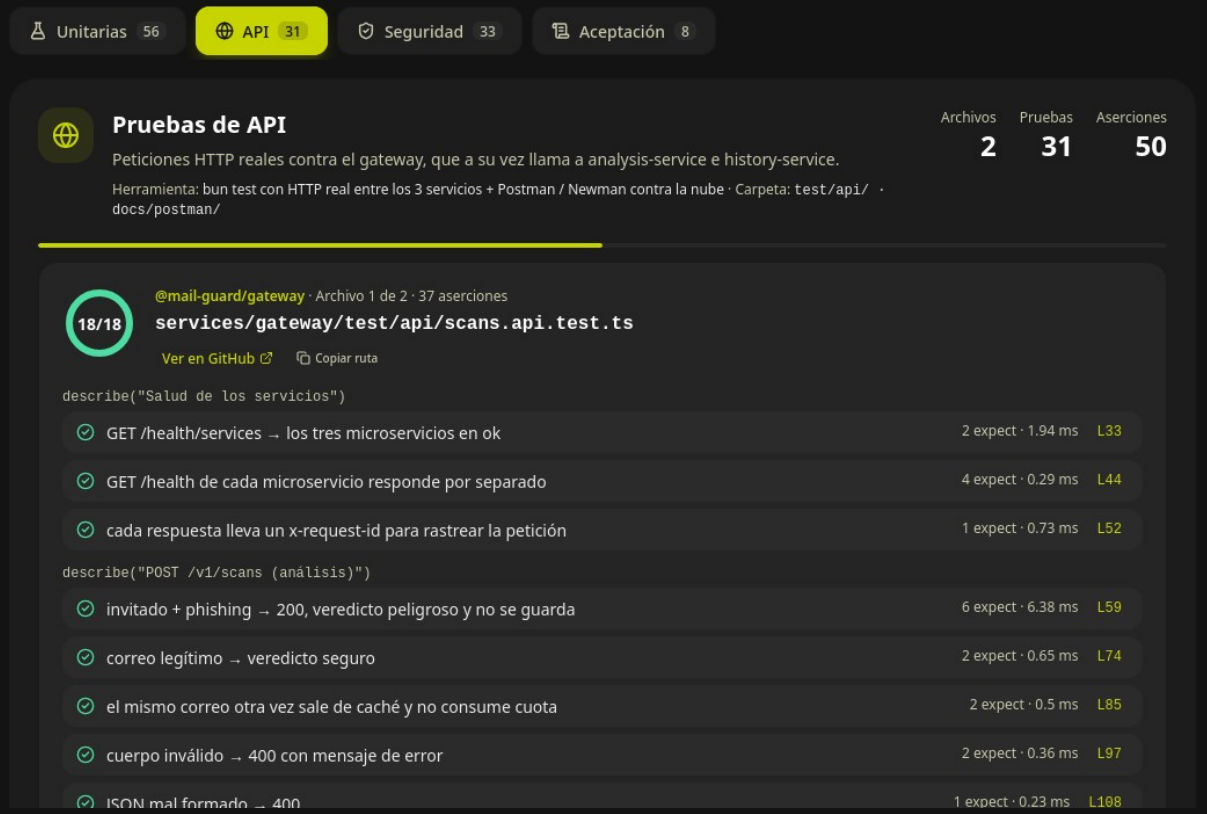
Pruebas de API · 31

18 pruebas automáticas · test/api/

- Se levantan los 3 microservicios reales y se hablan por HTTP
- Salud, análisis, caché, historial, perfil y borrado
- Errores: 400 cuerpo inválido, 404 ruta inexistente

Postman · 13 aserciones

- 6 peticiones contra el gateway real en Render (docs/postman/)
- Se importa en Postman o se corre con Newman
- La IA real (Jev) marcó el phishing como peligroso



Pruebas de seguridad · 33

Gateway · 14

Sin token o con JWT falso → 401 · Ana no ve ni borra lo de Luis · cuota → 429 · cuerpo grande → 413 · cabeceras y CORS

analysis-service · 7

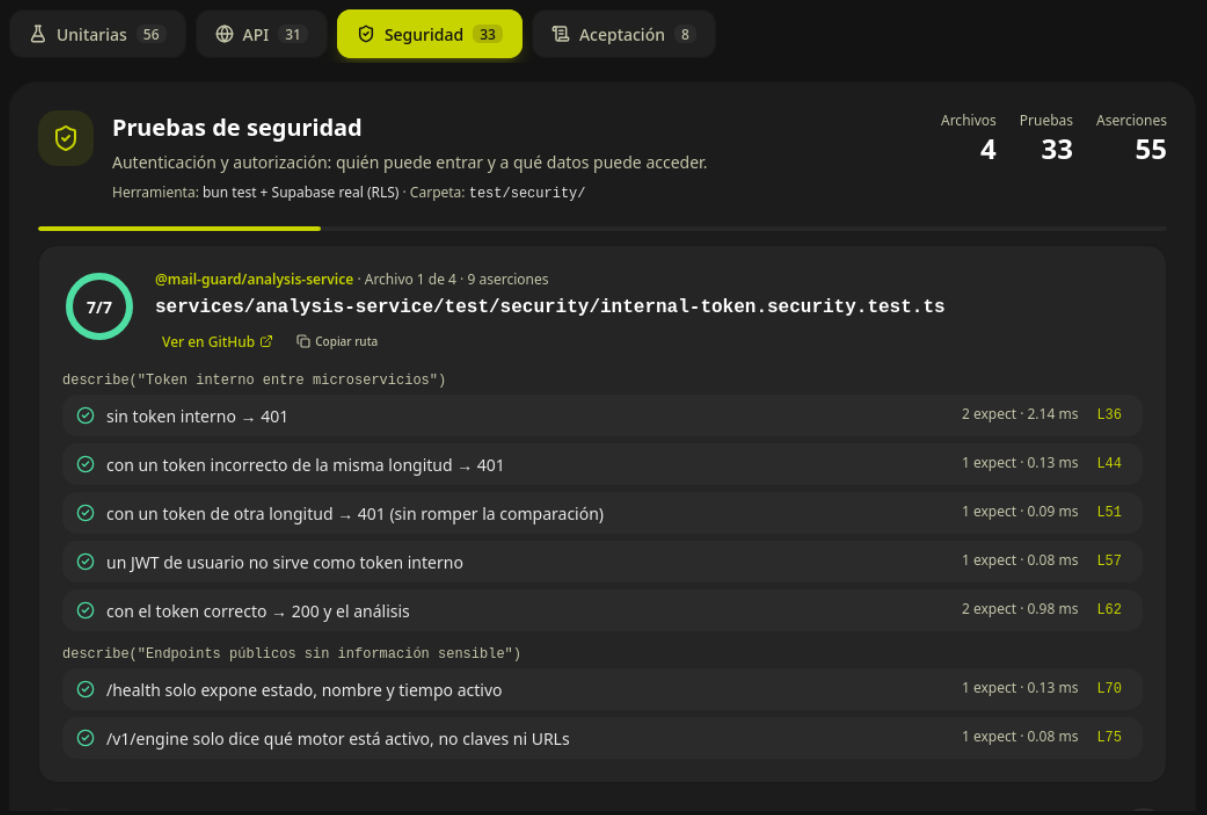
Sin token interno, o con uno incorrecto → 401 · /health no expone datos sensibles

history-service · 6

Doble llave: token interno + JWT · sin Supabase → 503 · id inválido → 404

Supabase real · 6

Un visitante anónimo no lee filas ni puede insertar · RLS activo · JWT falsificado rechazado



Lo que encontraron las pruebas

Bug corregido

Síntoma: un cuerpo de más de 512 KB respondía 500 «Error interno» en vez de 413.

Causa: el manejador de errores común solo reconocía AppError y convertía los errores HTTP de Hono en 500.

Arreglo: packages/service-kit/src/app.ts respeta el código HTTP. La prueba queda como regresión.

Riesgo identificado

Síntoma: un invitado puede saltarse la cuota diaria enviando una IP falsa en X-Forwarded-For. Comprobado en Render.

Impacto: análisis ilimitados sin cuenta y más costo de IA.

Recomendación: usar la IP que agrega el proxy de confianza (no la primera de la lista) y agregar la prueba de regresión.

Pruebas de aceptación · 8 escenarios

```
Característica: Historial de análisis (UC-07, UC-10, UC-13)

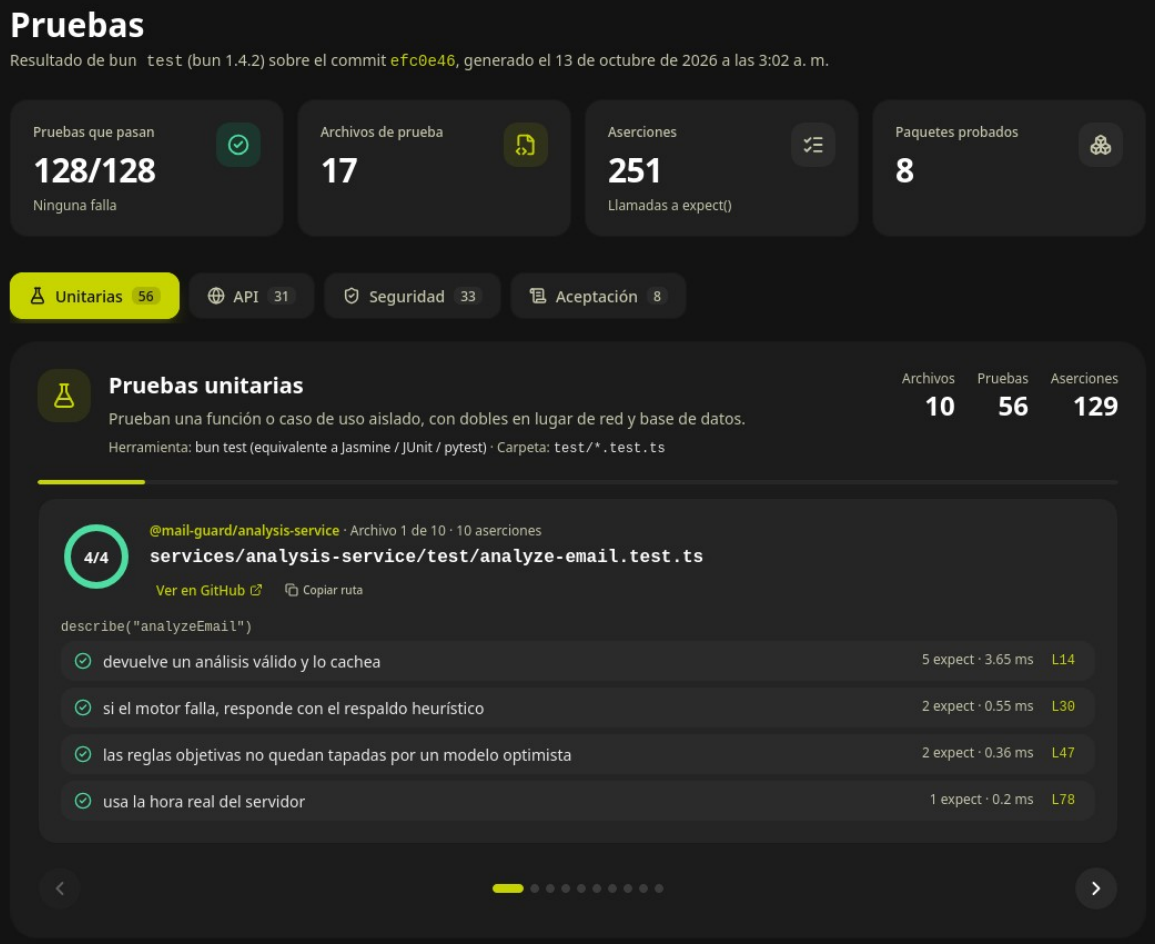
  Escenario: Borrar todo el historial
    Dado que Ana inició sesión
    Y que analizó el correo "phishing del banco"
    Y que analizó el correo "alerta legítima de Google"
    Cuando borra su historial
    Entonces se borraron 2 análisis
    Y su historial tiene 0 análisis
```

`services/gateway/test/acceptance/features/historial.feature`

8 de 8 escenarios pasan
17 verificaciones contra los 3 microservicios reales

- Escritos en español, como los casos de uso del documento USE_CASES.md
- Un runner sobre bun test ejecuta cada paso contra los 3 microservicios reales
- Si un paso falla, indica el archivo .feature y la línea exacta
- 2 características: analizar un correo y gestionar el historial

Evidencia verificable: /dashboard/tests



- Generada por bun run tests:report: ejecuta las pruebas de verdad y guarda el resultado con el commit
- Cada prueba enlaza a su línea exacta en GitHub (L14, L33…)
- Carrusel por tipo: unitarias, API, seguridad y aceptación
- Incluye los comandos para que cualquiera las vuelva a correr

Cómo ejecutarlas

`$ bun install`	Instala las dependencias del monorepo
`$ bun run test`	Corre las 115 pruebas locales de todos los paquetes
`$ cd services/gateway && bun test test/security`	Solo un tipo de prueba en un servicio
`$ bun run test:postman`	Colección de Postman contra Render
`$ bun run tests:report -- --postman`	Todo + actualiza la página de pruebas

Conclusiones

Tres microservicios con una sola responsabilidad cada uno, comunicados por HTTP y protegidos por un gateway.

128 pruebas de cuatro tipos, alineadas con las herramientas vistas en clase.

Las pruebas de seguridad comprueban autenticación, autorización y RLS, también contra la base real.

Probar sirvió: se encontró y corrigió un bug y se identificó un riesgo de abuso de cuota.

Próximos pasos

- Corregir la cuota por IP
- Restringir CORS a la extensión y al panel
- Pruebas E2E de la extensión con Selenium
- Ejecutar las pruebas en cada push (CI)

Mail Guard

¡Gracias! ¿Preguntas?

Código: github.com/bgrados31/uts-mail-guard

Pruebas: uts-mail-guard-extension-dashboard.vercel.app/dashboard/tests